

Spatial Joins with Grid-Based Indexing in Apache Spark

Panagiotis Athanasopoulos
Department of Digital Systems, University of Piraeus
Athens, Greece
me2402@webmail.unipi.gr

Abstract

This paper implements and evaluates spatial join algorithms using grid-based techniques in the distributed Apache Spark environment. The objective is to efficiently identify pairs of points from two datasets (RAILS and AREALM) that lie within a predefined distance threshold (ϵ). The main implementation stages, experimental methodology, and results are presented, with emphasis on the accuracy and efficiency of the approach for different parameter values. The experimental results show that the grid-based approach effectively exploits Spark's computational resources and exhibits good scalability on large-volume datasets. In addition, the main challenges encountered during the development and execution of the algorithms are discussed.

CCS Concepts

• Information systems → Spatial-temporal systems.

Keywords

Distributed data processing, Apache Spark, spatial joins, grid-based indexing, big data, algorithm performance

1 Introduction

In the era of big data, the need for efficient management and processing of geospatial data is becoming increasingly important [1]. Spatial queries, and spatial joins in particular, are among the fundamental tools for analyzing and extracting knowledge from geospatial datasets. However, the computational complexity of these operations increases substantially as the size and complexity of the data grow, making distributed processing systems such as Apache Spark necessary [6].

Spatial indexing based on grid-based methods offers a promising approach for accelerating spatial joins, since it partitions the space into smaller, manageable regions and thereby reduces the number of required comparisons [7]. This work studies the implementation and evaluation of a grid-based spatial join in Apache Spark, with the objective of improving performance and scalability when processing large volumes of geospatial data.

2 Problem Definition

This work addresses the efficient processing of spatial queries over large geospatial datasets by exploiting distributed processing in Apache Spark. Specifically, two datasets, R and S , are given, containing records that represent the geospatial locations of objects. Each record $r \in R$ (and correspondingly $s \in S$) is characterized by an object identifier (ID) and a position (x, y) , where x and y are geographic coordinates.

The objective is to implement algorithms that answer the following two queries:

- Query A: Find all pairs (r, s) for which the Euclidean distance $dist(r, s)$ is less than or equal to an input threshold ϵ . The output is the number of pairs satisfying the distance constraint.
- Query B: Find all records $r \in R$ that have more than k records $s \in S$ within distance ϵ . The value k is also an input parameter. The output is the subset of records r , together with the number of records s lying within distance ϵ .

To execute the above queries, a parallel algorithm is designed and implemented in Apache Spark using Euclidean distance as the distance metric. The real-world data are obtained from the RAILS (R) and AREALM (S) datasets, which are subsets of the TIGER data and are provided in tab-separated format.

3 Related Work

A substantial body of research has focused on the efficient implementation of spatial joins over large geospatial datasets, both in traditional database systems and in distributed infrastructures [4, 5]. Classical approaches rely on spatial indexes such as R-trees, quadrees, and grid-based structures in an effort to minimize the number of required comparisons [2].

At the distributed-systems level, several platforms and libraries have been proposed for implementing spatial operations while exploiting parallel processing, with prominent examples based on Hadoop and Spark [3, 7]. Each approach has advantages and disadvantages in terms of complexity, scalability, and ease of integration into large-scale applications.

The choice of grid-based indexing in this work is motivated by its implementation simplicity, its suitability for parallel processing, and its potential scalability compared with more complex indexing structures, particularly for real-world data with non-uniform distributions.

4 Methodology

Query A was implemented using parallel data processing in Apache Spark. The objective was to efficiently identify all point pairs (r, s) , where $r \in R$ and $s \in S$, whose Euclidean distance does not exceed the specified threshold ϵ .

Query A Implementation Steps

1. Data loading and transformation:

The two datasets (R and S) are loaded as RDDs, and each line is converted into a Point object containing the identifier (ID) and geographic coordinates (x, y) . The data are filtered so that invalid or incomplete records are ignored.

2. Construction of the grid-based index:

The geographic space is divided into equal-sized cells, where the

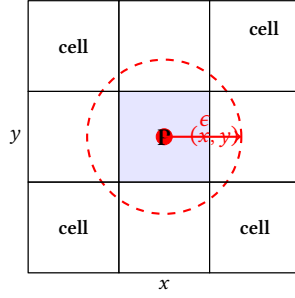


Figure 1: Example of grid-based partitioning: point (x, y) (red) belongs to the central cell, while the region of radius ϵ intersects the eight neighboring cells.

cell size is defined as $gridSize = \frac{\epsilon}{\sqrt{2}}$. This value is chosen so that candidate points whose Euclidean distance may be at most ϵ can be found in the same or neighboring cells.

Each point is assigned not only to its own cell but also to the eight neighboring cells, i.e., to all cells in the 3×3 grid around the point. This addresses border effects, in which point pairs near cell boundaries might otherwise not be examined if they fall into different cells.

3. Creation of key-value pairs and partitioning:

Each point is mapped to one or more cells through key-value pairs (cellKey, point). The data are partitioned using a Hash-Partitioner to distribute the workload and exploit Spark parallelism. Each partition can therefore process data associated with the same cells independently.

4. Join and filtering of candidate pairs:

A join is performed between the RDDs containing the points of R and S using the cell key. For each candidate pair associated with the same cell through the multi-cell assignment, the Euclidean distance is computed. Only pairs satisfying $dist(r, s) \leq \epsilon$ are retained. Additional filtering is applied when necessary to eliminate duplicate or symmetric pairs.

Algorithm 1 Basic spatial join algorithm with grid-based indexing

Require: Point sets R, S and distance threshold ϵ

Ensure: All pairs (r, s) such that $dist(r, s) \leq \epsilon$

```

1: Compute grid cell size:  $gridSize = \epsilon/\sqrt{2}$ 
2: for all point  $r$  in  $R$  do
3:   Assign  $r$  to all  $3 \times 3$  neighboring grid cells (based on
    $gridSize$ )
4: end for
5: for all point  $s$  in  $S$  do
6:   Assign  $s$  to all  $3 \times 3$  neighboring grid cells (based on
    $gridSize$ )
7: end for
8: For each cell:
9:   for all cell  $C$  do
10:    Retrieve  $R_C$  and  $S_C$ : points from  $R$  and  $S$  assigned to cell
     $C$ 
11:    for all  $r \in R_C$  do
12:      for all  $s \in S_C$  do
13:        if  $dist(r, s) \leq \epsilon$  then
14:          Output pair  $(r, s)$  as result
15:        end if
16:      end for
17:    end for
18: end for
```

5. Result storage:

The pairs (r, s) are stored as text, while for the purposes of the assignment only the number of pairs satisfying the distance criterion is returned. The final count is stored in a separate result file.

Advantages of the Approach

The implementation exploits Spark parallelism and reduces the number of comparisons through grid-based indexing. This avoids an exhaustive brute-force distance computation over every possible point pair, improving the efficiency and scalability of the algorithm. The selected grid size is intended to preserve valid candidate pairs, while partitioning distributes the processing workload across the available computational resources.

Query B Implementation Steps

Query B identifies all points $r \in R$ that have more than k neighbors $s \in S$ within distance ϵ . For each such point r , the required output is the number of neighboring points s satisfying the spatial criterion. Apache Spark and grid-based spatial partitioning are used to execute the query on large datasets.

1. Data loading and preprocessing: The data are loaded from HDFS as RDDs, and each line is converted into a Point object containing the identifier (ID) and coordinates (x, y) . Invalid or incomplete lines are then filtered out.

2. Grid-based indexing: The space is divided into equal-sized cells with side length $gridSize = \epsilon/\sqrt{2}$. To address border effects, each point is assigned both to its own cell and to its eight neighboring cells, forming a 3×3 cell neighborhood.

3. Grouping points by cell: The two datasets are mapped to key-value pairs using `cellKey` as the key, and a cogroup operation is then performed so that all r and s associated with each cell are grouped together.

4. Identification of candidate pairs (r, s) : For each cell, all possible (r, s) pairs are generated. The Euclidean distance is computed for each pair, and the pair is retained if the distance does not exceed ϵ .

5. Avoiding overcounting: Because the grid assignment may cause each (r, s) pair to appear multiple times through different cells, a distinct operation is applied so that each pair is counted only once.

6. Counting and filtering: Each r is associated with the number of distinct s points found within ϵ . Only those r points having more than k such neighbors are retained.

7. Result storage: The final output contains lines of the form $\{r.id, count\}$, where $count$ is the number of s points within ϵ .

The implementation avoids duplicate counting, exploits Spark parallelism, and supports the processing of large data volumes.

Algorithm 2 Basic Query B algorithm with grid-based indexing

Require: Point sets R, S ; parameters ϵ, k

Ensure: All $r \in R$ with $> k$ neighbors $s \in S$ such that $dist(r, s) \leq \epsilon$

- 1: Assign all points $r \in R$ and $s \in S$ to their grid cell and 8 neighbors
 - 2: For each cell, collect R_C, S_C (points from R, S in cell C)
 - 3: For all $r \in R_C, s \in S_C$:
 - 4: If $dist(r, s) \leq \epsilon$ add pair $(r.id, s.id)$
 - 5: Remove duplicate $(r.id, s.id)$ pairs
 - 6: For each r , count distinct s
 - 7: Filter r with $count > k$
 - 8: Output $\{r.id, count\}$
-

5 Experiments and Results

5.1 Experimental Procedure

The real-world RAILS and AREALM datasets were used to evaluate the implemented algorithm. They are subsets of the TIGER dataset and are provided in tab-separated format. The datasets contain 3,184,948 and 3,848,971 records, respectively.

The algorithm was executed with 200 partitions, as specified by the HashPartitioner during grid-based data partitioning. Different values of the distance threshold ϵ were tested.

5.2 Evaluation Metrics

The main evaluation metrics were the execution time for each ϵ value and the number of (r, s) pairs satisfying the spatial criterion. Execution time refers to the total time from job submission to completion, as reported by the YARN ResourceManager UI or by the wall-clock time of `spark-submit`.

5.3 Results

The Query A results show that as the distance threshold ϵ increases, both the number of detected (r, s) pairs and the execution time increase substantially. For small ϵ values (e.g., 0.003),

Table 1: Results for different values of ϵ

ϵ	Number of pairs	Execution time (sec)
0.012	41.927.160	900
0.008	18.819.585	511
0.006	10.602.167	469
0.003	2.739.782	317

fewer pairs are identified and execution time remains relatively low. For larger values (e.g., 0.012), the number of results grows sharply and the processing time becomes considerably higher.

This increase is expected because a larger ϵ expands the region examined around each point, producing more candidate pairs and requiring more distance comparisons. Consequently, the algorithm requires more computational resources and execution time.

Overall, there is a clear trade-off between the number of results and performance: increasing ϵ allows more spatial relationships to be detected but also increases the computational cost.

5.4 Discussion

The grid-based algorithm reduces the number of comparisons relative to a brute-force approach by exploiting partitioning and Spark parallelism. However, for large values of ϵ , the number of candidate pairs grows rapidly, negatively affecting response time. In addition, the selected number of partitions plays an important role in workload distribution.

5.5 Query B Experimental Procedure

The real-world RAILS and AREALM datasets, subsets of the TIGER dataset provided in tab-separated format, were used to evaluate Query B. RAILS contains 3,184,948 points, whereas AREALM contains 3,848,971 points.

The implementation was executed in Apache Spark using 200 partitions and grid-based partitioning with cell size $gridSize = \epsilon/\sqrt{2}$. The algorithm was run for different values of ϵ and k , and the results were measured in terms of the number of points r satisfying the spatial criterion and the job execution time.

5.6 Evaluation Metrics

The collected metrics were:

- Number of records r having more than k neighbors s within distance ϵ .
- Execution time for each parameter combination.

5.7 Results

The experimental results in Table 2 show that increasing ϵ substantially increases the number of records r having more than k neighbors within the ϵ distance. With larger ϵ values, each point r includes more points from S as neighbors, increasing both the number of r records satisfying the query and the required computation time because more point pairs must be processed. Conversely, smaller ϵ values reduce both the result size and execution time.

Table 2: Query B results for different values of ϵ and k

ϵ	k	Number of records r	Time (sec)
0.006	200	1.282	469
0.006	100	9696	525
0.003	200	115	349
0.003	100	515	342

The parameter k , which specifies the minimum number of neighbors required for a point r to be included in the output, directly affects the result size. Larger k values impose a stricter criterion and therefore fewer points r satisfy it, whereas smaller k values relax the filter. Overall, ϵ and k strongly influence both query efficiency and result volume, creating a trade-off between computational cost and result coverage.

6 Conclusions

This paper presented the implementation and evaluation of algorithms for efficiently executing spatial joins over large geospatial datasets using grid-based indexing techniques in Apache Spark. The experimental results demonstrate the behavior of the proposed method under different distance thresholds while preserving the required spatial-query results.

In particular, the number of results and the required processing time depend strongly on the distance parameter ϵ . For small ϵ values, the algorithm completes relatively quickly, whereas larger values produce a sharp increase in both the number of results and execution time because more candidate comparisons are generated.

Future work could focus on further performance optimization, for example through dynamic grid-size selection or more sophisticated spatial indexes such as R-trees or quadrees. Extending the implementation to support polygonal or spatio-temporal queries and evaluating it on even larger real-world datasets are also promising directions for future research.

Appendix A: Spark Job Execution

Execution command:

```
spark-submit --class org.example.Main --master
yarn \
--deploy-mode client --executor-memory 6g \
--num-executors 4 \
spark_project-1.0-SNAPSHOT.jar \
hdfs:///user/hduser/input/RAILS.csv \
hdfs:///user/hduser/input/AREALM.csv \
hdfs:///user/hduser/output/resultsA/ 0.012
```

Note: The results are stored in HDFS at the path specified by the output argument. Additional details about the jobs are included in the accompanying spark-jobs.txt file.

References

- [1] Ablimit Aji, Fusheng Wang, Hoang Vo, Rubao Lee, Qiaoling Liu, Xiaodong Zhang, and Joel Saltz. 2013. Hadoop-GIS: A High Performance Spatial Data Warehousing System over MapReduce. *Proceedings of the VLDB Endowment* 6, 11 (2013), 1009–1020. doi:10.14778/2536222.2536227
- [2] Norbert Beckmann, Hans-Peter Kriegel, Ralf Schneider, and Bernhard Seeger. 1990. The R*-Tree: An Efficient and Robust Access Method for Points and Rectangles. In *Proceedings of the 1990 ACM SIGMOD International Conference on Management of Data*. ACM, New York, NY, USA, 322–331. doi:10.1145/93597.98741
- [3] Ahmed Eldawy and Mohamed F. Mokbel. 2015. SpatialHadoop: A MapReduce Framework for Spatial Data. In *2015 IEEE 31st International Conference on Data Engineering*. IEEE, Piscataway, NJ, USA, 1352–1363. doi:10.1109/ICDE.2015.7113382
- [4] Michael R. Evans, Dev Oliver, Xun Zhou, and Shashi Shekhar. 2014. Spatial Big Data: Case Studies on Volume, Velocity, and Variety. In *Big Data: Techniques and Technologies in Geoinformatics*, Hassan A. Karimi (Ed.). CRC Press, Boca Raton, FL, USA, 149–176.
- [5] Tin Vu, Alberto Belussi, Sara Migliorini, and Ahmed Eldawy. 2024. A Learning-Based Framework for Spatial Join Processing: Estimation, Optimization and Tuning. *The VLDB Journal* 33 (2024), 1155–1177. doi:10.1007/s00778-024-00836-1
- [6] Simin You, Jianting Zhang, and Le Gruenwald. 2015. Large-Scale Spatial Join Query Processing in Cloud. In *2015 IEEE 31st International Conference on Data Engineering Workshops*. IEEE, Piscataway, NJ, USA, 34–41. doi:10.1109/ICDEW.2015.7129541
- [7] Jia Yu, Jinxuan Wu, and Mohamed Sarwat. 2015. GeoSpark: A Cluster Computing Framework for Processing Large-Scale Spatial Data. In *Proceedings of the 23rd SIGSPATIAL International Conference on Advances in Geographic Information Systems*. ACM, New York, NY, USA, Article 70, 4 pages. doi:10.1145/2820783.2820860